DESENVOLVIMENTO MOBILE · MÓDULO 01 · TRILHA KOTLIN · NÍVEL INICIANTE

Ambiente de Desenvolvimento

Android Studio, SDK, emulador, celular real, estrutura de projeto, Gradle e o primeiro app

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Instalar e configurar o Android Studio e o Android SDK.
- Executar apps no emulador e em um celular real (USB e Wi-Fi).
- Navegar pela estrutura de um projeto Android e entender cada arquivo.
- Entender o papel do Gradle, do *version catalog* e das variantes de build.
- Criar, executar e depurar o primeiro app com Jetpack Compose.
- Versionar o projeto com Git e GitHub.

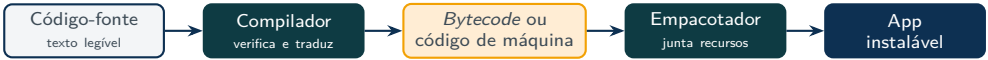
Roteiro

Neste módulo

- Fundamentos: ferramentas de desenvolvimento
- Ferramentas
- Executando o app
- Estrutura de um projeto
- Gradle
- Primeiro app e depuração
- Versionamento com Git
- Fechamento

Fundamentos: ferramentas de desenvolvimento

Do código-fonte ao programa



Compilador e interpretador

O **compilador** traduz o programa inteiro antes da execução e aponta erros cedo. O **interpretador** lê e executa o código aos poucos, durante a execução.

Máquina virtual

Programa que executa um *bytecode* intermediário em qualquer processador: a JVM e o ART (Kotlin) e a VM do Dart.

JIT e AOT

JIT: compila enquanto o programa roda; permite trocar código em execução (*hot reload*). **AOT**: compila tudo antes de distribuir; inicia rápido e roda com desempenho previsível.

Dica

As duas trilhas usam os dois: JIT para desenvolver com agilidade, AOT para o app que chega ao usuário (Módulo 15).

SDK, IDE e cadeia de ferramentas

SDK — *Software Development Kit*

Pacote com bibliotecas, compiladores, ferramentas de linha de comando, documentação e imagens de emulador para desenvolver para uma plataforma.

IDE — ambiente integrado

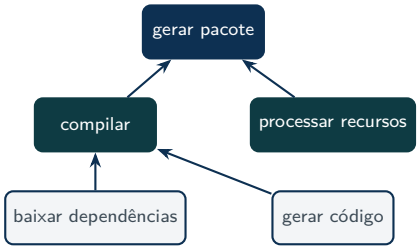
Editor que reúne escrita de código, compilação, execução, depuração, análise e integração com Git.

Analogia

O SDK é a caixa de ferramentas; a IDE é a bancada organizada onde você trabalha com elas.

	Trilha Kotlin	Trilha Flutter
Linguagem	Kotlin (sobre o JDK)	Dart (Dart SDK)
SDK da plataforma	Android SDK	Flutter SDK + Android SDK + Xcode
IDE	Android Studio	VS Code ou Android Studio
Build	Gradle	`flutter` CLI (usa Gradle e Xcode)
Aparelho	ADB	ADB e `flutter devices`
Pacotes	Maven Central, Google Maven	pub.dev

Sistemas de *build* e dependências



Grafo de tarefas: cada uma só roda depois das que ela usa.

Sistema de *build*

Ferramenta que transforma o projeto em app executando **tarefas** organizadas num grafo de dependências. Só refaz o que mudou (*build* incremental) e guarda resultados em cache.

Dependência transitiva

A biblioteca que você usa depende de outras, que dependem de outras. O gerenciador resolve a árvore inteira e escolhe versões compatíveis.

Atenção

Fixe as versões com arquivos de trava (catálogo de versões, `pubspec.lock`) para que todos na equipe compilem o mesmo app.

Emuladores, simuladores e aparelhos reais

	Vantagem	Limite
Emulador Android	vários tamanhos e versões, GPS e sensores simulados	sem Bluetooth, NFC e USB reais
Simulador iOS	rápido, integrado ao Xcode	roda código do Mac; não imita o hardware
Aparelho real	desempenho e hardware verdadeiros	cada modelo exige um aparelho

Emulador e simulador

O **emulador** imita o aparelho, inclusive o processador e o sistema completo. O **simulador** apenas reproduz o comportamento do sistema, executando o app compilado para o computador.

Por que o emulador exige virtualização

O emulador Android roda uma imagem completa do sistema numa máquina virtual. Com aceleração por hardware (Intel VT-x, AMD-V, KVM no Linux, Hypervisor no macOS), as instruções rodam direto no processador e o emulador fica rápido.

Depuração

Depurador

Ferramenta que pausa o programa num ponto escolhido (*breakpoint*) para inspecionar variáveis, a pilha de chamadas e executar passo a passo.

Comando	O que faz
Step over	executa a linha e para na próxima
Step into	entra na função chamada
Step out	termina a função atual e volta
Resume	continua até o próximo *breakpoint*

Método científico da depuração

1. Reproduza o problema de forma confiável.
2. Formule uma hipótese sobre a causa.
3. Teste a hipótese (*breakpoint*, *log*, teste automatizado).
4. Corrija e escreva um teste que impeça a volta do defeito.

Dica

Leia a mensagem de erro e a pilha de chamadas inteiras: a primeira linha do **seu** código costuma apontar a causa.

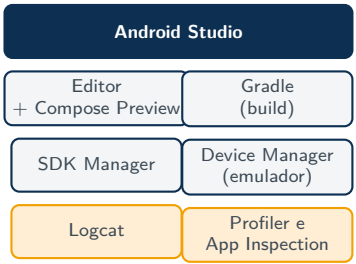
Ferramentas

Android Studio

- IDE oficial do Android, baseada no IntelliJ IDEA (JetBrains).
- Gratuita, para Windows, macOS, Linux e ChromeOS.
- Inclui o JDK, o SDK Manager, o emulador, o *profiler* e o inspetor de layout.
- Assistente de IA integrado (Gemini).

Requisitos práticos

- **RAM:** 16 GB; com 8 GB, prefira o celular real.
- **Disco:** SSD com 20 GB livres.
- **CPU:** virtualização ativada na BIOS.



Instalação passo a passo

1. Baixe o instalador em `https://developer.android.com/studio`.
2. **Windows:** execute o `.exe`. **Linux:** extraia o `.tar.gz` em `/opt` e rode `bin/studio.sh`. **macOS:** arraste para Aplicativos.
3. No assistente de primeira execução escolha **Standard**: ele baixa o SDK, as ferramentas de build e uma imagem do emulador.
4. Abra **Settings** → **Languages & Frameworks** → **Android SDK** e confira os pacotes.

Pacotes essenciais do SDK

- **SDK Platform** da API alvo (ex.: Android 16).
- **Build-Tools** e **Platform-Tools** (contém o `adb`).
- **Android Emulator** e uma *system image*.

Linux: aceleração do emulador

Instale o KVM e adicione seu usuário ao grupo `kvm`. Sem aceleração por hardware o emulador fica lento demais para uso.

Executando o app

Emulador (AVD)

AVD (*Android Virtual Device*) é uma configuração de aparelho virtual: modelo, tela, versão do Android e memória.

1. **Device Manager** → + (Create Virtual Device).
2. Escolha um perfil (ex.: *Pixel*, *Medium Phone*).
3. Escolha uma *system image* com **Google Play** ou **Google APIs**.
4. Prefira imagens **x86_64** (PC) ou **arm64** (Mac Apple Silicon).

> **Dica**
> O emulador simula GPS, rotação, bateria, rede lenta e sensores em *Extended Controls* (...).



Celular real: depuração USB e Wi-Fi

Ativando as opções do desenvolvedor

1. **Configurações** → **Sobre o telefone**.
2. Toque 7 vezes em **Número da versão**.
3. Em **Opções do desenvolvedor**, ative **Depuração USB**.
4. Conecte o cabo e aceite a impressão digital RSA no celular.

Depuração por Wi-Fi (Android 11+)

1. Celular e computador na mesma rede.
2. Ative **Depuração por Wi-Fi**.
3. No Android Studio: **Device Manager** → **Pair devices using Wi-Fi** e leia o QR code.

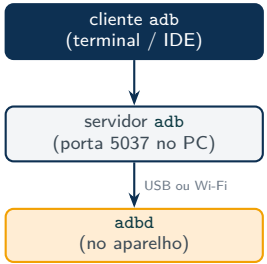
Aparelho real é obrigatório para

Bluetooth/BLE, NFC, câmera de verdade, sensores físicos e medições realistas de desempenho e bateria. Projetos com ESP32 (Módulo 13) **exigem** celular real.

ADB: *Android Debug Bridge*

Comandos úteis

```
adb devices                  # lista aparelhos conectados
adb install app-debug.apk    # instala um APK
adb uninstall com.exemplo.app
adb logcat -s LabMonitor     # filtra logs pela tag
adb shell                    # terminal no aparelho
adb shell pm list packages | grep exemplo
adb push foto.jpg /sdcard/Download/
adb pull /sdcard/Download/log.txt .
adb shell screencap -p /sdcard/tela.png
adb reverse tcp:8080 tcp:8080  # localhost do PC
```



Dica

`adb reverse` permite que o app no celular acesse uma API rodando no seu computador.

Estrutura de um projeto

Criando o projeto

1. **File** → **New** → **New Project**.
2. Modelo **Empty Activity** (já vem com Compose).
3. Preencha:
   - **Name**: LabMonitor
   - **Package name**: `br.edu.ifma.labmonitor`
   - **Minimum SDK**: API 26
   - **Build configuration language**: Kotlin DSL
4. Aguarde o **Gradle Sync** terminar.

Package name / applicationId

Identificador **único e permanente** do app na Google Play. Use o domínio invertido da instituição ou o seu. Depois de publicado, **não pode ser alterado**.

Dica

Use a visualização **Android** no painel de projeto para ver a estrutura lógica, e **Project** para ver as pastas reais no disco.

Árvore de diretórios

LabMonitor/

```
LabMonitor/
|-- app/
|   |-- build.gradle.kts      (módulo)
|   |-- proguard-rules.pro
|   `-- src/
|       |-- main/
|       |   |-- AndroidManifest.xml
|       |   |-- java/.../labmonitor/
|       |   |   |-- MainActivity.kt
|       |   |   `-- ui/theme/
|       |   `-- res/
|       |       |-- drawable/  mipmap-*/
|       |       `-- values/strings.xml
|       |-- test/             (unitários)
|       `-- androidTest/      (instrumentados)
|-- gradle/
|   |-- libs.versions.toml    (catálogo)
|   `-- wrapper/
|-- build.gradle.kts           (projeto)
|-- settings.gradle.kts
|-- gradle.properties
`-- gradlew, gradlew.bat
```

- `app/`: o módulo do aplicativo. Projetos grandes têm vários módulos.
- `src/main/java`: código Kotlin (o nome da pasta é histórico).
- `res/`: recursos que não são código: imagens, textos, ícones, cores.
- `test/`: testes que rodam na JVM do computador.
- `androidTest/`: testes que rodam no aparelho.
- `gradlew`: *wrapper* que baixa a versão certa do Gradle. Sempre use-o, inclusive na integração contínua.

Dica

Os textos da interface ficam em `strings.xml`. Isso facilita a tradução do app.

AndroidManifest.xml

app/src/main/AndroidManifest.xml

```
<manifest xmlns:android=
    "http://schemas.android.com/apk/res/android">

  <uses-permission
      android:name="android.permission.INTERNET" />

  <application
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name"
      android:theme="@style/Theme.LabMonitor">
    <activity
        android:name=".MainActivity"
        android:exported="true">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER"
      />
      </intent-filter>
    </activity>
  </application>
</manifest>
```

O manifesto declara ao sistema:

- as **permissões** necessárias;
- o **ícone**, o **nome** e o **tema**;
- os **componentes** (Activities, Services...);
- qual Activity abre pelo **ícone do launcher** (`MAIN` + `LAUNCHER`).

Atenção

`android:exported="true"` é obrigatório em componentes com `intent-filter`.

MainActivity.kt: o ponto de entrada

MainActivity.kt

```
class MainActivity : ComponentActivity() {
  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()  // desenha atrás das barras
    setContent {        // a partir daqui é Compose
      LabMonitorTheme {
        Scaffold(Modifier.fillMaxSize()) { padding ->
          Saudacao(
            nome = "Laboratório",
            modifier = Modifier.padding(padding)
          )
        }
      }
    }
  }
}

@Composable
fun Saudacao(nome: String, modifier: Modifier = Modifier) {
  Text(text = "Olá, $nome!", modifier = modifier)
}
```

- `ComponentActivity`: Activity base para Compose.
- `onCreate`: chamado quando a tela é criada (Módulo 07).
- `setContent {}`: define a interface com funções `@Composable`.
- `LabMonitorTheme`: tema Material 3 gerado em `ui/theme/`.
- `Scaffold`: estrutura básica de tela (barras, FAB).

Dica

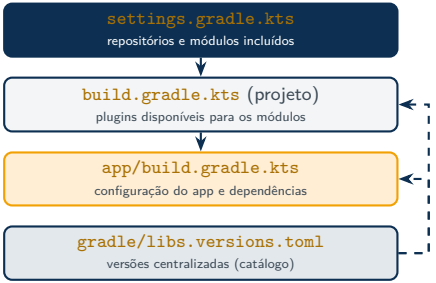
Os detalhes de Compose ficam para o Módulo 05. Por enquanto, rode e altere o texto.

Gradle

O que é o Gradle

Gradle é o sistema de build: ele baixa dependências, compila, empacota, assina e roda testes.

- Os scripts são escritos em **Kotlin DSL** (`.gradle.kts`).
- O **Android Gradle Plugin (AGP)** ensina o Gradle a construir apps Android.
- **Gradle Sync**: a IDE lê os scripts e configura o projeto. Rode-o após qualquer mudança nos arquivos de build.



Version catalog e dependências

gradle/libs.versions.toml

```
[versions]
agp = "..."              # versões atuais
kotlin = "..."           # o assistente
composeBom = "..."       # preenche

[libraries]
androidx-compose-bom = { group = "androidx.compose",
  name = "compose-bom", version.ref = "composeBom" }
androidx-material3 = { group = "androidx.compose.material3",
  name = "material3" }

[plugins]
android-application = { id = "com.android.application",
  version.ref = "agp" }
```

app/build.gradle.kts

```
plugins {
  alias(libs.plugins.android.application)
  alias(libs.plugins.kotlin.compose)
}
android {
  namespace = "br.edu.ifma.labmonitor"
  compileSdk = 36
  defaultConfig {
    applicationId = "br.edu.ifma.labmonitor"
    minSdk = 26
    targetSdk = 36
    versionCode = 1
    versionName = "1.0"
  }
}
dependencies {
  implementation(platform(libs.androidx.compose.bom))
  implementation(libs.androidx.material3)
  testImplementation(libs.junit)
}
```

BOM (*Bill of Materials*): fixa versões compatíveis de todas as bibliotecas Compose.

Configurações de dependência, tarefas e variantes

Configuração	Uso
`implementation`	código do app
`testImplementation`	testes unitários
`androidTestImplementation`	testes no aparelho
`debugImplementation`	só no build de debug
`ksp`	processadores de anotação

Build types

debug: depurável, assinado com chave de teste. **release**: otimizado (R8), assinado com a sua chave, vai para a loja.

Tarefas pelo terminal

```
./gradlew tasks              # lista tarefas
./gradlew assembleDebug      # gera APK debug
./gradlew installDebug       # instala no aparelho
./gradlew test               # testes unitários
./gradlew lint               # análise estática
./gradlew bundleRelease      # gera AAB (loja)
./gradlew clean              # limpa o build
```

Dica

Os APKs gerados ficam em `app/build/outputs/apk/`.

Primeiro app e depuração

Compose Preview e execução

Preview

```
@Preview(showBackground = true, name = "Claro")
@Preview(
  uiMode = Configuration.UI_MODE_NIGHT_YES,
  name = "Escuro"
)
@Composable
fun SaudacaoPreview() {
  LabMonitorTheme {
    Saudacao("Bancada 3")
  }
}
```

Dica

O *Preview* renderiza a função na IDE sem instalar o app. Use vários `@Preview` para comparar tema claro e escuro, fontes grandes e telas diferentes.

Executando

1. Escolha o aparelho na barra superior.
2. Clique em **Run** (▶) ou **Shift+F10**.
3. O Gradle compila, instala e abre o app.

Live Edit

Com o app rodando, mudanças em funções Compose aparecem no aparelho sem reinstalar (*Live Edit*).

Logcat e depurador

Registrando logs

```
private const val TAG = "LabMonitor"

fun lerSensor(valor: Double) {
  Log.d(TAG, "Leitura recebida: $valor")
  if (valor > 80.0) {
    Log.w(TAG, "Temperatura alta: $valor")
  }
}
```

`Log.v`	verbose	detalhes extremos
`Log.d`	debug	desenvolvimento
`Log.i`	info	eventos normais
`Log.w`	warn	situação suspeita
`Log.e`	error	falhas

Depurador

- Clique na margem para criar um *breakpoint*.
- Rode com **Debug** (ícone de inseto).
- Inspecione variáveis, avalie expressões e avance linha a linha (F8/F7).

Filtro do Logcat

`package:mine tag:LabMonitor level:debug` mostra só os logs do seu app.

Atenção

Nunca registre senhas, tokens ou dados pessoais em logs.

Outras ferramentas da IDE

Ferramenta	Para que serve
Layout Inspector	Ver a árvore de componentes do app em execução e contar recomposições
App Inspection	Inspecionar o banco de dados (Room), o tráfego de rede e os *workers* em tempo real
Profiler	Medir CPU, memória e energia; achar travamentos (Módulo 14)
Device Explorer	Navegar pelos arquivos do aparelho
Running Devices	Espelhar e controlar o celular físico dentro da IDE
Gemini no Android Studio	Gerar código, explicar erros e sugerir testes (sempre revise o resultado)

Atalhos que valem ouro

Shift Shift: busca global a definição **Alt+Enter**: correções rápidas **Ctrl+Alt+L**: formatar **Ctrl+B**: ir para

Versionamento com Git

Git e GitHub no projeto

Fluxo básico

```
git init
git add .
git commit -m "Projeto LabMonitor criado"
git branch -M main
git remote add origin
      https://github.com/<usuario>/LabMonitor.git
git push -u origin main
# a cada etapa:
git checkout -b feature/lista-equipamentos
git commit -am "Tela de lista de equipamentos"
```

O que NÃO versionar

O `.gitignore` gerado pelo Android Studio já exclui:

- `build/`, `.gradle/`, `.idea/` (parcial);
- `local.properties` (caminho do SDK na sua máquina).

Atenção

Nunca faça commit de **chaves de API**, **keystores** (`.jks`) ou senhas. Se isso acontecer, considere a chave comprometida e gere outra.

Fechamento

Resumo

Resumo

- Android Studio = editor + SDK + emulador + Gradle + ferramentas de inspeção.
- Rode apps no emulador (AVD) ou no celular real com depuração USB/Wi-Fi; o `adb` faz a ponte.
- `AndroidManifest.xml` declara componentes e permissões; `MainActivity.kt` chama `setContent`.
- O Gradle constrói o app; o *version catalog* centraliza versões e o BOM alinha as bibliotecas Compose.
- `minSdk`, `targetSdk` e `applicationId` ficam no `app/build.gradle.kts`.
- Logcat, depurador e Layout Inspector são as ferramentas de diagnóstico do dia a dia.

Glossário

Termo	Significado
SDK	conjunto de bibliotecas e ferramentas para desenvolver para uma plataforma
IDE	ambiente integrado: editor, *build*, execução e depuração
Gradle	sistema de *build* do Android, baseado em tarefas e *plugins*
Catálogo de versões	arquivo `libs.versions.toml` que centraliza as dependências
AVD	aparelho virtual configurado no emulador do Android
ADB	ferramenta de linha de comando que conversa com aparelhos e emuladores
Logcat	registro de mensagens do sistema e do app
Breakpoint	ponto em que o depurador pausa a execução
Manifesto	`AndroidManifest.xml`: componentes, permissões e metadados do app

Questões de revisão

1. Qual a diferença entre compilar com JIT e com AOT? Onde cada um aparece no Android?
2. O que o Android SDK contém além das bibliotecas?
3. Por que o Gradle consegue recompilar só parte do projeto?
4. O que é uma dependência transitiva e por que ela pode causar conflitos?
5. Por que o emulador precisa de aceleração por hardware?
6. Cite duas situações em que o emulador não substitui um aparelho real.
7. Para que servem o ADB e o Logcat?
8. Descreva o método para depurar um app que fecha ao tocar num botão.

Exercícios

Exercício 1 — Ambiente

Crie dois AVDs: um celular com a API mais recente e um com API 26. Rode o app nos dois e anote as diferenças visuais.

Exercício 2 — ADB

Pelo terminal, instale o APK de debug no seu celular, capture uma tela com `adb shell screencap` e copie a imagem para o computador.

Exercício 3 — Gradle

Altere `versionName` para `"1.0-aula"`, gere o APK com `./gradlew assembleDebug` e localize o arquivo gerado. O que acontece se `minSdk` for maior que a API do seu emulador?

LabMonitor — Etapa 1

Crie o projeto LabMonitor (`br.edu.ifma.labmonitor`), troque o texto inicial por "LabMonitor — DEE/IFMA", adicione um `Log.i` no `onCreate` e publique o repositório no GitHub.

Para aprofundar

- Conheça o Android Studio: `https://developer.android.com/studio/intro`
- Rodar apps em aparelho físico: `https://developer.android.com/studio/run/device`
- Emulador: `https://developer.android.com/studio/run/emulator`
- ADB: `https://developer.android.com/tools/adb`
- Configurar o build: `https://developer.android.com/build`
- Version catalogs: `https://developer.android.com/build/migrate-to-catalogs`

Respostas das questões de revisão

1. JIT compila durante a execução; AOT, antes. O ART combina os dois: compila AOT partes do app (com perfis) e usa JIT no restante.
2. Ferramentas de *build* e de linha de comando (ADB), imagens de sistema para o emulador, documentação e as plataformas de cada versão da API.
3. Porque organiza o trabalho em tarefas com entradas e saídas conhecidas: se as entradas não mudaram, reaproveita o resultado (*build* incremental e cache).
4. É a dependência de uma dependência. Duas bibliotecas podem exigir versões diferentes da mesma biblioteca; o gerenciador precisa escolher uma compatível.
5. Porque executa um sistema completo numa máquina virtual; com VT-x, AMD-V ou KVM as instruções rodam direto no processador.
6. Uso de Bluetooth, NFC ou USB; medição de desempenho e bateria; câmera e sensores reais.
7. O ADB instala, inicia e depura apps e abre um terminal no aparelho; o Logcat mostra as mensagens e a pilha de erros.
8. Reproduzir o fechamento, ler a pilha no Logcat, colocar um *breakpoint* antes da linha suspeita, testar a hipótese, corrigir e criar um teste.

Fim do Módulo 01

Próximo: Módulo 02 — Kotlin essencial: tipos, null safety, funções, classes e coleções.